

# EcoSort: Intelligent Recycling Sorting System

ICT304 Assignment • Developing an AI System • 3 October 2026

Valliammai Nivetha Alagappan, AI and Data Lead, 35556565

Robert Alexander Chee Ying Sheng, Architecture and Integration Lead, 35998337

Waitun, Evaluation, Dashboard and Reporting Lead, 36056783

## 1. ABSTRACT

EcoSort is an intelligent recycling sorting system for the point of disposal. A camera photographs a waste item, a classifier assigns it to one of six recycling categories, and the item is routed to the correct bin. Items the classifier is unsure about are sent to a person for review instead of being misrouted, and every decision is logged to a live contamination dashboard. This report covers the first stages of the system engineering life cycle: problem formulation, requirements, system and sub-system design, and a working prototype of the AI sub-system. The prototype compares two classification techniques on identical held-out data: a support vector machine on hand-made image features, and MobileNetV2 transfer learning. MobileNetV2 reached 0.832 test accuracy and 0.817 macro-F1, and it is the recommended primary technique.

## 2. Introduction

Kerbside recycling contamination, the presence of incorrect or non-recyclable items in recycling streams, is a persistent and costly problem. When contaminated batches reach Materials Recovery Facilities (MRFs), entire loads can be rejected and diverted to landfill, even when most of the material was correctly sorted. At the point of disposal, most sorting decisions are made by individuals guessing at category rules, and at the facility level, sorting remains largely manual and error-prone.

EcoSort is proposed as an intelligent recycling sorting system that addresses this problem directly at the point of disposal. The system captures a photograph of a waste item, classifies it into one of six recycling categories using a trained AI model, routes it to the correct bin, and, critically, flags items the model is uncertain about for human review rather than risking a silent misclassification. Every decision the system makes is logged to a live contamination dashboard, giving facility operators real-time visibility into sorting accuracy and contamination trends.

The system is deliberately designed as a product, not merely a classification model. It combines an AI sub-system, the waste classifier, with supporting non-AI sub-systems: image capture, pre-processing, decision routing, logging, dashboard, user interface and simulated actuation. Together they form one end-to-end pipeline, suitable for a low-cost public recycling station or an MRF infeed point.

## 3. System Engineering Process and Life Cycle

The project follows the NASA Systems Engineering Handbook product life cycle, cross-referenced with the INCOSE and ISO 15288 phase names. Progress is reviewed with the tutor every week, as the brief requires. This assignment covers the first stages, Pre-Phase A

through Phase B: problem formulation, requirements definition, system and sub-system design, and a working prototype of the AI sub-system. The later phases belong to the final project.

| NASA phase                              | INCOSE phase           | Key tasks                                                                | Covered in    |
|-----------------------------------------|------------------------|--------------------------------------------------------------------------|---------------|
| Pre-Phase A, Concept Studies            | Concept                | Problem formulation, literature, stakeholder needs                       | Assignment    |
| Phase A, Concept and Technology         | Concept/Development    | Requirements (EXP/REQ), choice of techniques to investigate, data source | Assignment    |
| Phase B, Preliminary Design             | Development            | System architecture, sub-system design, AI sub-system prototype          | Assignment    |
| Phase C, Final Design and Fabrication   | Development/Production | Build all sub-systems, refine the chosen technique                       | Final project |
| Phase D, Assembly, Integration and Test | Production             | Integrate, run the test plan, tune                                       | Final project |
| Phase E, Operations and Sustainment     | Utilization/Support    | Demo, evaluation, deployment and maintenance plan, user guide            | Final project |
| Phase F, Closeout                       | Retirement             | Final report, presentation, retrospective                                | Final project |

## 4. Requirements

Following the stakeholder-expectation-to-requirement approach used in this unit, the core requirement was derived as follows.

**Stakeholder: recycling facility operator / council waste-management team.**

**EXP-01 (stakeholder expectation):**

“The operator needs the system to automatically classify each waste item into the correct recycling category from a photo, route it to the right bin, and show running contamination statistics, so that sorting is faster and more consistent than manual inspection, without silently misrouting uncertain items.”

**REQ-01 (technical requirement, derived from EXP-01):**

“The system shall classify an item image into one of six classes: glass, paper, cardboard, plastic, metal, trash, with test accuracy  $\geq 85\%$  (target, actual result reported), per-class precision/recall/F1 and a confusion matrix on a held-out test set, inference  $\leq 1$  second per item on commodity hardware, and shall route items with confidence below a threshold  $\theta$  to a human-review queue, logging every decision to a dashboard.”

| Attribute           | Value                                                                                 |
|---------------------|---------------------------------------------------------------------------------------|
| ID                  | REQ-01                                                                                |
| Derived from        | EXP-01                                                                                |
| Verification method | Analysis (cross-validation + held-out test set) + Demonstration (end-to-end run)      |
| Acceptance          | Accuracy $\geq 0.85$ ; per-class F1 reported; latency $\leq 1$ s/item; low-confidence |

|          |                                                                             |
|----------|-----------------------------------------------------------------------------|
| criteria | routing functions correctly; dashboard reflects logged decisions accurately |
|----------|-----------------------------------------------------------------------------|

The accuracy target is a goal to confirm against actual results rather than a hard pass/fail bar. The unit brief does not require results to exceed published literature, only that they are understood, reported honestly and justified.

## 5. System Overview

EcoSort is built from seven sub-systems. Six are ordinary software and hardware components; one is the AI sub-system. The brief asks for both kinds to be identified.

| Sub-system               | AI or non-AI | Build, buy or reuse      | Purpose                           |
|--------------------------|--------------|--------------------------|-----------------------------------|
| Image Capture            | non-AI       | Buy/reuse (webcam/phone) | Acquire a photo of the item       |
| Pre-processing           | non-AI       | Build                    | Normalise / extract features      |
| Waste Classifier         | AI           | Build (train from data)  | Class label + confidence          |
| Decision and Bin-Routing | non-AI       | Build                    | Route to a bin or to human review |
| Logging and Dashboard    | non-AI       | Build                    | Running contamination statistics  |
| User Interface           | non-AI       | Build                    | Show the result to the user       |
| Simulated Actuation      | non-AI       | Build/simulate           | Indicate the correct bin          |

Workflow: an item is placed in front of the camera, photographed, pre-processed and classified. The classifier returns a label and a confidence score. If the confidence is at or above the threshold, the item is routed to the correct bin; if it is below, the item goes to a human-review queue. Every decision is logged and the dashboard updates. Training is an offline sub-process; the deployed system runs inference only.

## 6. Test Plan

The test plan covers the sub-systems one at a time (unit level) and the system as a whole (integration and system level), matching the sub-system breakdown above.

| Level | Component tested         | Test method                                 | Pass criterion                                                                                        |
|-------|--------------------------|---------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Unit  | Image Capture            | Manually trigger capture from webcam/upload | Returns a valid image of expected shape/format                                                        |
| Unit  | Pre-processing           | Feed sample images through the pipeline     | Output shape correct; normalised means $\approx 0$ , scale $\approx 1$ (SVM feature path)             |
| Unit  | Waste Classifier (AI)    | Run on held-out test set                    | Accuracy $\geq 0.85$ ; per-class precision/recall/F1 reported; confusion matrix produced              |
| Unit  | Decision and Bin-Routing | Feed known class + confidence pairs         | Correct class $\rightarrow$ bin mapping; confidence $< \theta$ correctly routes to human-review queue |
| Unit  | Logging and Dashboard    | Run a batch of decisions through the logger | Dashboard totals match logged totals exactly (no drift/miscount)                                      |

|                 |                           |                                                                 |                                                                                                                                            |
|-----------------|---------------------------|-----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Integrati<br>on | End-to-end<br>pipeline    | Run a batch of held-<br>out images through<br>the full pipeline | Each item reaches the correct bin (or<br>review queue), is logged, and is shown<br>correctly in the UI                                     |
| System          | Full system<br>under load | Run a larger batch<br>continuously                              | Inference latency $\leq 1$ s/item;<br>dashboard updates correctly reflect<br>the full run; threshold behaviour holds<br>under repeated use |

AI-specific evaluation: the two candidate techniques, SVM on hand-crafted features and MobileNetV2 transfer learning, are evaluated on identical metrics: accuracy, per-class precision/recall/F1, confusion matrix and inference latency. This justifies the final technique choice against the REQ-01 acceptance criteria.

Test data: the stratified 70/15/15 train/validation/test split (random\_state = 42) keeps the test set unseen during training and tuning, so the held-out results measure genuine generalization.

## 7. AI Technique Investigation

EcoSort's AI sub-system is a six-class image classifier. It returns a predicted class and a confidence score. Two techniques were built on the same data and evaluated.

### Technique A: SVM on hand-crafted features

The SVM (Cortes & Vapnik, 1995) classifies an item from measurements the team designed. Each photo becomes a single vector with three families of features:

- HOG (histogram of oriented gradients) captures the shape of the outline, following Dalal and Triggs (2005): 16x16 pixel cells, 2x2 cell blocks, at 128x128 resolution.
- A colour histogram with 32 bins per RGB channel captures the colour distribution.
- GLCM texture features (contrast, dissimilarity, homogeneity, energy, correlation) at distances 1 and 2 and four angles capture surface roughness, which separates cardboard from plastic film.

The vector is 1,900 numbers long. The pipeline is StandardScaler followed by SVC(kernel='rbf', probability=True). Scaling sits inside a scikit-learn Pipeline, so it is fitted on training data only, which prevents leakage (Pedregosa et al., 2011). probability=True matters because the routing sub-system needs a confidence score, not just a label.

Settings came from a small grid search with 5-fold cross-validation, scored on macro-F1: C in {1, 10} and gamma in {scale, 0.01}. The best combination was C=10, gamma=scale. Feature extraction for the 1,768 training images took 22.2 seconds; the grid search took 181.3 seconds.

### Technique B: MobileNetV2 transfer learning

The second classifier starts from MobileNetV2 pre-trained on ImageNet (Sandler et al., 2018) and adapts it to the six classes.

- Backbone frozen; the head replaced with global average pooling, dropout 0.2 and a dense softmax layer with six outputs.
- Input resized to 224x224. The earlier runs used 128x128 to keep the notebook fast on a laptop; the larger input is what lifted the test accuracy from 0.776 to 0.832.
- Training-time augmentation only: horizontal flip, rotation 0.1, brightness 0.1, zoom 0.1.
- Class weights computed from the training distribution, so trash receives 3.07 times the weight of an average class.

- Adam optimiser, sparse categorical cross-entropy, batch size 32, up to 12 epochs with early stopping (patience 4).

Training took 125.6 seconds on a T4 GPU. The best validation accuracy was 0.836, reached in epoch 11 of 12.

## Confidence for routing

Both techniques expose a confidence score: the maximum class probability from `predict_proba` for the SVM, and the softmax maximum for MobileNetV2. In the finished system, decisions below a confidence threshold go to a person instead of a bin. That design choice is what makes a partly opaque model acceptable in the workflow, and it is why confidence was built into the pipeline from the first version.

## 8. Data

Source: TrashNet (Thung & Yang, 2016), the 512x384 version, loaded from the Hugging Face Hub.

| Class     | Photos |
|-----------|--------|
| Cardboard | 403    |
| Glass     | 501    |
| Metal     | 410    |
| Paper     | 594    |
| Plastic   | 482    |
| Trash     | 137    |
| Total     | 2,527  |

The class distribution is the first real constraint. Trash is 5.4 percent of the data, and a model tuned for raw accuracy can simply ignore it. Three choices address that: a stratified split, class weights in the CNN loss, and per-class reporting so the weak class stays visible.

Split: stratified 70/15/15 with `random_state=42`, the same split for both techniques and for every run.

- 1,768 photos to train
- 379 to validate during training
- 380 held back for the final test

The ratios come out the same in each split. Paper, for example, sits at roughly 0.235 in training, validation and test, so the rare class stays rare everywhere.

The three sets have separate jobs. The training set fits the models. The validation set tunes settings such as the confidence threshold. The test set is used once, at the end, to report final results. Keeping the test set untouched during tuning means the reported figures reflect how the system performs on images it has not seen.

### Pre-processing:

- Both paths convert to RGB and resize. The SVM path resizes to 128x128 for feature extraction. The CNN path resizes to 224x224 and applies MobileNetV2's `preprocess_input`.
- The SVM's scaler is fitted inside a Pipeline on training data only.
- The CNN's augmentation applies at training time only; validation and test images pass through unchanged.

Reproducibility: seeds are fixed for numpy and TensorFlow, so the same images land in the same split on every run. The dataset downloads itself from the notebook, about 3.6 GB the first time. A copy of the resized dataset and the pretrained weights also sits on Kaggle, so the same notebook runs offline on a Kaggle GPU.

## 9. Evaluation Method

This section explains how EcoSort is evaluated against its goal. The goal comes from requirement REQ-01: the system must classify a photo of a waste item into one of six categories, route it to the right bin, send uncertain items to a person for review, and record every decision on a contamination dashboard. Evaluation therefore has two parts. The first part judges the AI sub-system, meaning the waste classifier. The second part judges the whole system working together.

### Evaluation goals

Each element of REQ-01 is matched to one measurement below, so every claim in the results traces back to a requirement.

| REQ-01 element                             | What we measure                                         | How                                                  |
|--------------------------------------------|---------------------------------------------------------|------------------------------------------------------|
| Classification accuracy $\geq$ 85%         | Overall accuracy on the held-out test set               | Predictions vs. true labels, test split only         |
| Per-class precision / recall / F1          | Quality for each of the six classes, including trash    | Per-class report + macro-averaged F1                 |
| Confusion matrix                           | Which classes are mistaken for which                    | Matrix on the test split, both techniques            |
| Inference $\leq$ 1 s per item              | Time to classify one image                              | Mean over repeated runs                              |
| Route low-confidence items to human review | Effect of the threshold on accuracy and review workload | Threshold sweep on the validation set                |
| Log every decision to a dashboard          | Dashboard totals equal log totals                       | Logging and Dashboard unit test, then end-to-end run |

### What is being compared

The brief asks for at least two techniques to be investigated, with a justification for the one chosen. Both candidates from the design document were evaluated:

- Technique A: a Support Vector Machine (SVM) on hand-crafted features: HOG (histogram of oriented gradients), a colour histogram and GLCM (grey-level co-occurrence matrix) texture, preceded by feature scaling.
- Technique B: a MobileNetV2 convolutional network fine-tuned by transfer learning.

One thing changes between the two runs: the technique itself. Data, split, metrics and hardware stay the same, so any difference in results can be credited to the technique and not to the setup. The one deliberate exception is data augmentation, which applies to the CNN only, because the SVM works on fixed hand-crafted features. This is stated so it is not mistaken for an unfair advantage.

### Metrics and why they were chosen

Accuracy alone would be misleading for this dataset. Trash makes up only about 5.4% of the images, so a model that ignored that class completely could still score above 90% accuracy. Per-class measures are therefore reported next to overall accuracy.

| Metric                | What it tells us                                    | Why it suits EcoSort                                                                                    |
|-----------------------|-----------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Accuracy              | Share of all items classified correctly             | Directly matches the REQ-01 target, but on its own hides weak classes                                   |
| Precision (per class) | Of the items sent to a bin, how many belonged there | Low precision means that bin is being contaminated, which is the exact problem EcoSort exists to reduce |
| Recall (per           | Of the items that belong in                         | Low recall means recyclable material is                                                                 |

|                     |                                                                       |                                                                                                       |
|---------------------|-----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| class)              | a bin, how many reached it                                            | being lost to the wrong bin                                                                           |
| F1 and macro-F1     | Balance of precision and recall; macro-F1 weights every class equally | Stops the large classes from hiding poor performance on trash                                         |
| Confusion matrix    | Full pattern of mix-ups between classes                               | Shows whether errors are the harmless kind (paper vs. cardboard) or the costly kind (glass vs. trash) |
| Inference latency   | Seconds to classify one item                                          | Checks the real-time requirement of $\leq 1$ s per item                                               |
| Low-confidence rate | Share of items sent to human review                                   | Measures the workload the human-in-the-loop design creates                                            |

Macro-averaged F1 is the main figure for comparing the two techniques, because it gives the small trash class the same weight as the large classes. Accuracy is still reported because it is the figure named in REQ-01.

### Experimental procedure

- Fit both techniques on the training set, using the validation set for tuning and early stopping.
- Freeze all settings, then score each technique once on the test set and record accuracy, per-class precision, recall and F1, macro-F1 and the confusion matrix.
- Measure inference latency on a single image at a time, after a short warm-up, averaged over repeated runs. Hardware: Kaggle notebooks, 2 x T4 GPU (measured means: 2.7 ms for the SVM and 3.7 ms for MobileNetV2).
- Compare the two techniques on identical metrics and recommend one, weighing accuracy, macro-F1, latency and model size.

The test set holds only 20 trash images, so one extra mistake changes that class's recall by about 5 percentage points. The trash figures are treated as indicative, and this is said in the Discussion.

### Confidence threshold experiment

The human-review mechanism depends on the confidence threshold  $\theta$ : anything below it goes to a person, anything above it goes to a bin. Following the brief's instruction to vary one factor at a time,  $\theta$  is swept from 0.3 to 0.9 on the validation set, recording two things for each value: the share of items sent to human review, and the accuracy on the items the system accepts automatically. A higher  $\theta$  means fewer wrong items reach the bins, but more items go to a person. The chosen  $\theta$  is the one that keeps automatic accuracy high without overloading the review queue, justified by the resulting curve.  $\theta$  only changes routing, not the classifier itself, so the technique results below stand regardless of the sweep. The value used in the logging prototype (0.60) is a placeholder until the sweep is complete.

### System-level evaluation

Once the sub-systems are integrated, the integration and system tests from the test plan run on held-out images. Each image passes through capture, pre-processing, classification, routing, logging and the dashboard. The checks: every item reaches the correct bin or the review queue, every decision is logged, dashboard totals equal log totals, and end-to-end latency stays within 1 second per item. The dashboard reports two live indicators: the low-confidence rate, which needs no ground truth, and the contamination rate, which is measured on evaluation runs where the true class is known.

## Acceptance criteria

The results are judged against the REQ-01 acceptance criteria below. The accuracy target is a goal to confirm rather than a strict pass mark, in line with the brief: results need not beat the published literature, but they must be understood, reported honestly and justified.

| Acceptance criterion (REQ-01)              | Target                        | Result                                                                                | Met?                                                                                         |
|--------------------------------------------|-------------------------------|---------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| Test accuracy of chosen technique          | $\geq 0.85$                   | 0.832 (MobileNetV2; SVM 0.711)                                                        | No, just under target: 0.832 against 0.85. The Discussion covers the gap and the next steps. |
| Per-class F1 and confusion matrix reported | Reported                      | Macro-F1 0.817 (MobileNetV2), 0.703 (SVM); matrices in outputs/                       | Yes                                                                                          |
| Inference latency                          | $\leq 1$ s per item           | 3.7 ms per image (MobileNetV2); 2.7 ms (SVM)                                          | Yes, by a wide margin                                                                        |
| Low-confidence items routed to review      | conf < $\theta$ goes to queue | The classifier returns a confidence for every item; the review queue is not built yet | Partly: confidence measured, routing not built yet                                           |
| Dashboard totals equal log totals          | Exact match                   | Passed in the logging unit test (200 of 200 entries)                                  | Yes, at unit level; end-to-end run pending                                                   |

## 10. Results

All numbers below are from the recorded run of 2 October 2026 (SVM 0.711, MobileNetV2 0.832). The confusion matrices are in outputs/cm\_SVM.png and outputs/cm\_MobileNetV2.png. At 128x128 the CNN moved between 0.755 and 0.803 across repeated runs; at 224x224 it reached 0.832. The SVM returned 0.711 every time.

| Measure               | Technique A: SVM                | Technique B: MobileNetV2          | Note                            |
|-----------------------|---------------------------------|-----------------------------------|---------------------------------|
| Test accuracy         | 0.711                           | 0.832                             | Target $\geq 0.85$              |
| Macro-F1              | 0.703                           | 0.817                             |                                 |
| F1 on trash           | 0.67                            | 0.70                              | Minority class, 20 test images  |
| Mean latency (s/item) | 0.0027                          | 0.0037                            | Target $\leq 1.0$               |
| Model size            | Feature-based; no large weights | About 9 MB of MobileNetV2 weights | Relevant to low-cost deployment |

### Per-class results

- SVM on hand-crafted features: accuracy 0.711, macro-F1 0.703. Per-class F1: cardboard 0.76, glass 0.69, metal 0.67, paper 0.80, plastic 0.64, trash 0.67.
- MobileNetV2 transfer learning: accuracy 0.832, macro-F1 0.817. Per-class F1: cardboard 0.90, glass 0.82, metal 0.83, paper 0.86, plastic 0.79, trash 0.70.

## Where it errs

Glass is still the hardest class: its recall sits at 0.74, and the remaining mix-ups fall between glass, plastic and cardboard. Trash improved to F1 0.70 but stays the smallest class. Paper and cardboard separate cleanly. The training curves (outputs/mobilenetv2\_training.png) show accuracy rising and loss falling until the curves flatten. The one-photo demo (outputs/demo\_prediction.png) shows a held-back metal photo classified correctly by both techniques, with confidences 0.62 for the SVM and 0.85 for MobileNetV2. The notebook also carries an upload step, so a reader can classify their own photo and see both predictions side by side with the routing decision.

## Against the requirement

REQ-01 set a target of 85 percent test accuracy. The measured 0.832 falls just short, and it is reported as measured. Raising the input to 224x224 moved the CNN from 0.776 to 0.832; the remaining gap traces to the small dataset and the rare classes, not to the approach. Latency: SVM 2.7 milliseconds per photo, MobileNetV2 3.7 milliseconds per photo, measured on the test set. Both sit far inside the one-second limit from the requirement.

## Running the prototype

The prototype is one notebook, submission/EcoSort\_AI\_Prototype.ipynb, also public at <https://www.kaggle.com/code/nivalagappan/ecosort-ai-sub-system-prototype>. It downloads the dataset and the pretrained weights itself and runs end to end in about eight minutes on a free Kaggle GPU. A supporting notebook, EcoSort\_Logging\_Dashboard.ipynb, implements the logging and dashboard module with its unit tests.

# 11. Comparison and Justification

MobileNetV2 is the stronger classifier on this data, and it is the technique recommended as the primary. It leads by 12.1 accuracy points and 11.4 macro-F1 points. The reason is the data. With 1,768 training photos, hand-crafted features only encode what the team thought to measure: outline, colour, texture. The pre-trained network brings features learned from more than a million images and adapts them to the six classes. That is also why a from-scratch CNN was rejected (Seetharaman, 2025): it would need more data than is available and would risk overfitting.

Two honest points on the comparison. First, the rare trash class stays the hardest for both techniques: F1 0.67 (SVM) against 0.70 (CNN) on just 20 test images, and the class moves across runs. Second, speed does not settle the choice: both answer in about 3 milliseconds per photo, far inside the one-second requirement.

The real trade-off is interpretability. The SVM's decisions sit on measurements a person can inspect. The CNN's do not. The system design bounds that opacity: the CNN exposes a confidence score, and low-confidence items go to human review instead of being trusted blindly.

The SVM stays in the project as a lightweight, interpretable fallback for hardware that cannot run TensorFlow, and as a reference point that shows what transfer learning adds on identical data.

# 12. Discussion

## What went well

Transfer learning delivered what the literature predicts: on 1,768 training photos the pre-trained network beat the hand-featured baseline by 12.1 accuracy points and 11.4 macro-F1

points. The pipeline is reproducible, the split is fixed, and the whole notebook runs end to end in about eight minutes on free hardware. Inference costs about 3 milliseconds per photo, so the classifier will not be the deployment bottleneck. Confidence scores exist for both techniques, which unblocks the human-review routing the system design depends on.

### What went poorly

The 85 percent target was not met, though the gap narrowed from 0.776 to 0.832 once the input rose to 224x224. Glass recall still sits at 0.74: transparent, thin and glossy items stay the hardest to place. The trash class remains small and unstable across runs, which is what 137 training examples predict.

### What comes next, in order of expected value

- Fine-tune the top layers of the backbone after the head converges, at a low learning rate.
- Collect more trash photos, even a few hundred, to stabilise the rare class.
- Calibrate the routing threshold against the confidence distribution, which the notebook's outputs already provide, so review rates and error rates can be traded deliberately.
- Add test-time augmentation at inference for a small accuracy gain at the cost of speed.

## 13. Conclusion

MobileNetV2 transfer learning is the technique recommended for the EcoSort classifier: it is the most accurate of the two on identical data, fast enough for a live bin, and its confidence scores make the human-review fallback workable. The SVM remains a valid lightweight alternative and a useful reference point. The remaining weaknesses are data problems more than model problems, and the improvement path is concrete.

## 14. Plan and Milestones

The team meets the tutor every week, as the brief requires. The milestones below map the remaining work to the life cycle phases.

| Week                   | NASA phase            | Tasks                                                                                                       | Milestone                             |
|------------------------|-----------------------|-------------------------------------------------------------------------------------------------------------|---------------------------------------|
| Wk 1<br>(Sep 14–20)    | Pre-Phase A / Phase A | Concept, literature, stakeholder needs, requirements draft, tutor approval of the requirement; set up tools | M1: requirement approved by tutor     |
| Wk 2<br>(Sep 21–27)    | Phase A / B           | Load TrashNet, EDA and imbalance check, build SVM baseline, start MobileNetV2                               | M2: data ready + SVM baseline results |
| Wk 3<br>(Sep 28–Oct 3) | Phase B               | System design document, finish MobileNetV2, run comparison, initial results; submit Assignment (3 Oct)      | M3: Assignment submitted              |
| Wk 4<br>(Oct 5–11)     | Phase C               | Build the non-AI sub-systems (capture, routing, logging, dashboard)                                         | M4: sub-systems built                 |
| Wk 5<br>(Oct 12–18)    | Phase D               | Integrate and run the test plan, tune                                                                       | M5: integrated system tested          |
| Wk 6<br>(Oct 19–25)    | Phase D/E             | Evaluation, deployment and maintenance plan, user guide draft                                               | M6: plan + guide drafted              |

|                            |              |                                               |                                          |
|----------------------------|--------------|-----------------------------------------------|------------------------------------------|
| Wk 7<br>(Oct 26–<br>Nov 1) | Phase E      | Demo video, presentation slides, final report | M7: demo + slides<br>done                |
| Wk 8<br>(Nov 2–<br>7)      | Phase<br>E/F | Finalise and submit Final Project (7 Nov)     | M8: Final<br>submitted +<br>presentation |

## 15. Tools Used

The brief requires all four tool categories to be reported. The team reused the tools set up earlier in the unit.

| Category           | Tool            | Use here                                         |
|--------------------|-----------------|--------------------------------------------------|
| Project Management | Trello          | Sprint board, milestones M1–M8                   |
| Version Control    | GitHub (Git)    | Code, notebook, report; branches + pull requests |
| Collaboration      | Google Drive    | Shared report, slides, figures                   |
| Communication      | Microsoft Teams | Weekly tutor check-ins, stand-ups                |

Some tools overlap, and this is stated clearly: GitHub also supports collaboration and communication through issues and pull requests; Teams also supports collaboration through files and a wiki; Trello also supports light collaboration through card comments.

Trello is the project management tool. The EcoSort board holds eight milestones, one per weekly sprint, and each milestone is one project deliverable. Tasks sit on cards, and the board tracks progress, shows upcoming work, and records whether each weekly milestone was completed.

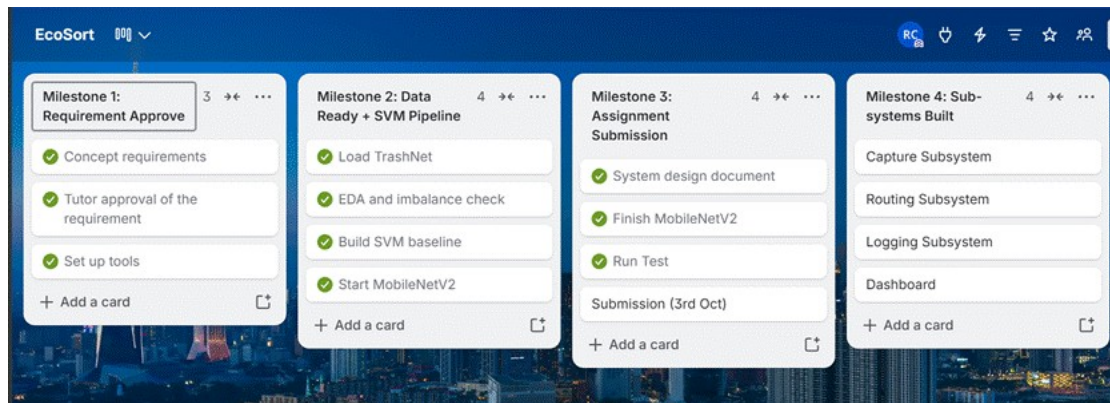


Figure 1. First-Half of milestones and tasks

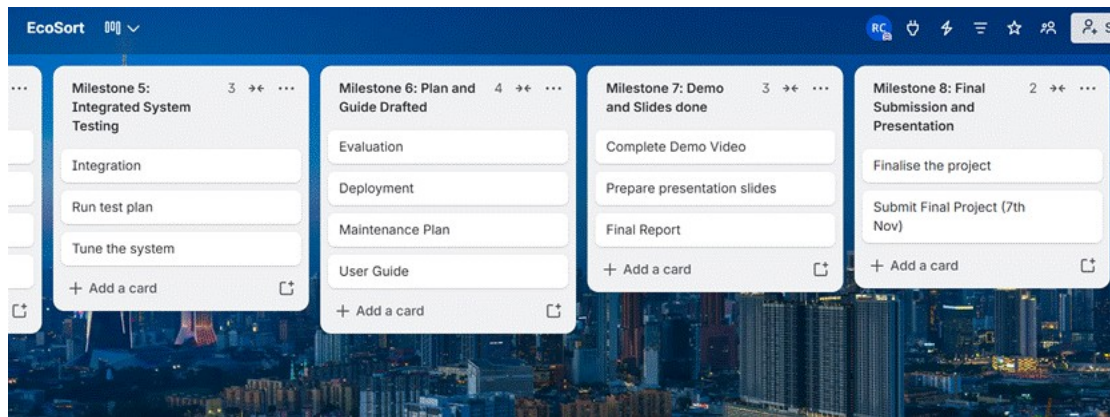


Figure 2. Second-Half of milestones and tasks

GitHub is the version control platform. A shared repository stores the EcoSort source code and project files, so each member contributes through branches and pull requests while one central copy stays current.

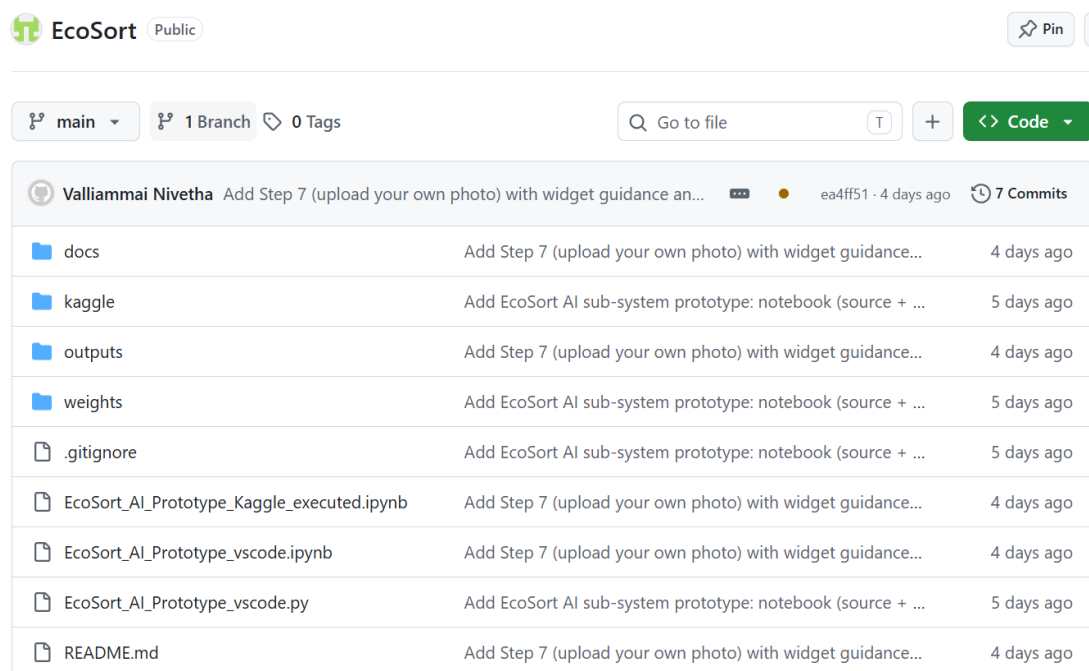


Figure 3. The files and codes in GitHub

All project files live in one shared Google Drive folder, "ICT304 Assignment 1". Both teammates have Editor access, so everyone can open, edit and comment on the same copy of each file. Documents are drafted and reviewed there before the final upload; code stays on GitHub and tasks stay on Trello.

| Tool          | Google Drive                                               |
|---------------|------------------------------------------------------------|
| Shared folder | ICT304 Assignment 1                                        |
| Access        | Owner plus two Editors (shared with each teammate's Google |

|                  |                                                                                                                                                                           |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                  | account)                                                                                                                                                                  |
| What lives there | AI prototype notebook (EcoSort_AI_Prototype.ipynb), draft report (EcoSort_Report_DRAFT.docx), submission notes (EcoSort_Submission_Notes.docx), project page (index.html) |
| Purpose          | Single shared copy of the documents for drafting and review before submission                                                                                             |



Figure 4. The shared folder, "ICT304 Assignment 1".

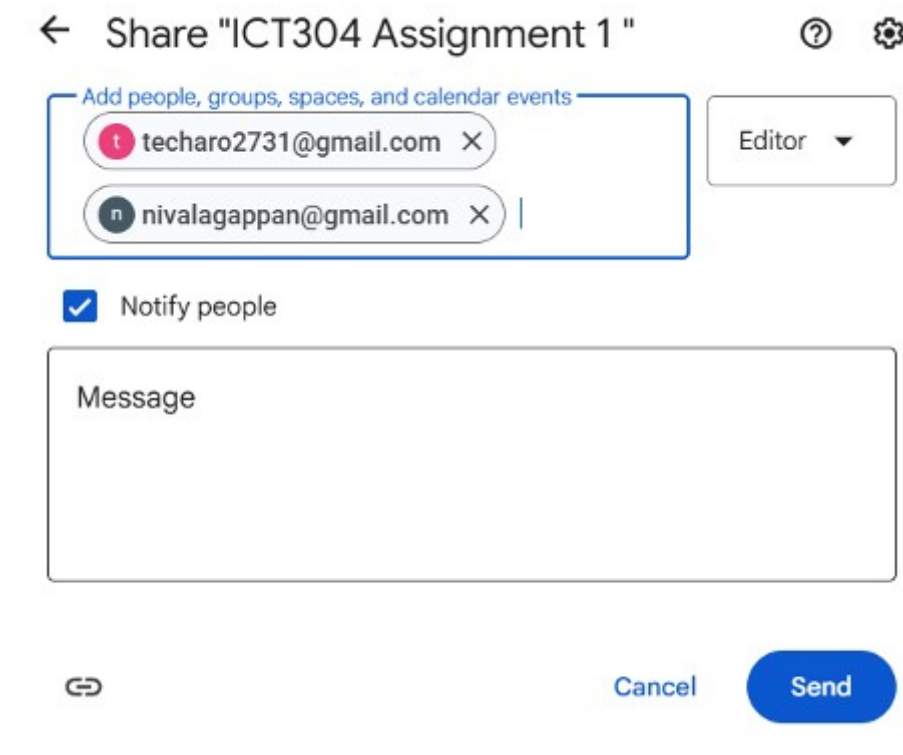


Figure 5. Sharing settings: both teammates added with Editor access.

|                               |    |            |        |  |
|-------------------------------|----|------------|--------|--|
| EcoSort_AI_Prototype.ipynb    | me | 7:23 PM me | 960 KB |  |
| EcoSort_Report_DRAFT.docx     | me | 7:23 PM me | 33 KB  |  |
| EcoSort_Submission_Notes.docx | me | 7:23 PM me | 8 KB   |  |
| index.html                    | me | 7:23 PM me | 39 KB  |  |

Figure 6. Files stored in the folder (notebook, draft report, submission notes and project page).

Microsoft Teams is the communication tool. The team meets the tutor there every week, as the brief requires, and uses the same channel for stand-up updates between meetings.

## 16. References

- Anthropic. (2026). Claude Sonnet 5 [Large language model]. <https://claude.ai>
- Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297. <https://doi.org/10.1007/BF00994018>
- Dalal, N., & Triggs, B. (2005). Histograms of oriented gradients for human detection. In 2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05) (Vol. 1, pp. 886–893). IEEE. <https://doi.org/10.1109/CVPR.2005.177>
- Haralick, R. M., Shanmugam, K., & Dinstein, I. (1973). Textural features for image classification. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-3(6), 610–621. <https://doi.org/10.1109/TSMC.1973.4309314>
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (pp. 770–778). IEEE. <https://doi.org/10.1109/CVPR.2016.90>
- INCOSE. (2023). *INCOSE systems engineering handbook* (5th ed.). Wiley.
- ISO/IEC/IEEE. (2015). *Systems and software engineering—System life cycle processes* (ISO/IEC/IEEE 15288:2015). International Organization for Standardization.
- NASA. (2016). *NASA systems engineering handbook* (NASA/SP-2016-6105 Rev2). National Aeronautics and Space Administration.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. In 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition (pp. 4510–4520). IEEE. <https://doi.org/10.1109/CVPR.2018.00474>
- Seetharaman, G. (2025). CNN waste classification with OpenCV and PyTorch [Source code]. GitHub. <https://github.com/gokulseetharaman/cnn-waste-classification-opencv-pytorch>
- Swain, M. J., & Ballard, D. H. (1991). Color indexing. *International Journal of Computer Vision*, 7(1), 11–32. <https://doi.org/10.1007/BF00130487>
- Thung, G. (2016). TrashNet [Data set]. Hugging Face. <https://huggingface.co/datasets/garythung/trashnet>
- Thung, G., & Yang, M. (2016). Classification of trash for recyclability status [Project report]. Stanford University.
- van der Walt, S., Schönberger, J., Nunez-Iglesias, J., Boulogne, F., Warner, J. D., Yager, N., Gouillart, E., Yu, T., & the scikit-image contributors. (2014). scikit-image: Image processing in Python. *PeerJ*, 2, Article e453. <https://doi.org/10.7717/peerj.453>

## 17. AI-Tool Declaration

Generative AI was used in the preparation of this report, as permitted by the unit information guide. The team used Claude (Anthropic, 2026) to debug and migrate the prototype notebook, run it on Kaggle, build the project page, and draft report sections. Every AI output was reviewed, tested and edited by the team, and each member is responsible for the material submitted. The prompts used are attached to the submission.